

TripMate AI: Intelligent Group Activity Planning Platform

Powered by a Unified Deep-Reasoning Agent

Project Idea, Features & Technical Architecture Proposal for Supervisor

Project Team (6 Members): Ta Quang Huy Ha Dang Huy Pham Dinh Anh Duong Nguyen Minh Duc
Nguyen Tung Duong Dinh Tien Luan

1. Introduction & Project Idea

1.1. The Problem

Organizing activities for a group of 3–10+ people (a trip, a dinner, a weekend hangout) is painful in practice:

- **Manual research** across maps, booking sites and reviews takes days.
- **Schedule overlap** across members with different free windows ($\pm\delta$ tolerance) is a combinatorial puzzle.
- **Conflicting preferences** (beach vs. mountain, budget vs. comfort) lead to decision paralysis.
- **Fragmented chat** on Zalo/Messenger buries plans and links.
- **One organizer** carries the entire load and gets blamed if the group is unhappy.
- **Expense splitting** after the trip is tedious and awkward.

1.2. The Solution

TripMate AI turns group planning into a guided, consensus-driven flow in under 15 minutes: members chat freely in a shared room → an AI agent asks clarifying questions → the agent researches and proposes an itinerary → the group votes leg-by-leg → the plan is locked and expenses are auto-settled.



Figure 1: End-to-end collaborative planning workflow

2. Platform Features

- **Group Room:** Shared chat room, invite by link. Members chat freely — budget, dietary needs, dates and preferences are shared naturally in conversation or collected by the agent when it detects they are needed. 6 event types: TRAVEL, DINING, HANGOUT, ENTERTAINMENT, SIGHTSEEING, CUSTOM. Roles: Host, Member, Viewer.
- **AI Planning Hub:** One-click itinerary generation, live flight/hotel/ticket lookup, dietary-aware dining suggestions, visa/passport checks.
- **Voting & Customization:** Leg-by-leg UP/DOWN voting with instant alternatives, live AI chat to tweak the plan, manual drag-and-drop editor.
- **Map & Utilities:** Interactive map (Mapbox), weather-based packing checklist, PDF export, Vietnamese/English UI.
- **Expense Splitting:** Tracks advances and payments, supports equal/exact/percentage splits, computes minimum peer-to-peer settlements.

3. The AI Agent: Tools, Skills & Operational Flow

3.1. Model Layer

The agent is powered by **DeepSeek-V4**, used in two modes rather than two separate agents:

- **Flash** — fast chat, room-message parsing, tool-call formatting.
- **Pro** — deep reasoning for constraint solving and preference conflicts.

One agent, one shared context, two model tiers chosen per sub-task by complexity.

3.2. Tools (external capabilities)

Each tool is a typed function the agent can call. This is the concrete contract, not just a diagram box:

Tool	Purpose & key inputs → output	Provider
<code>search_places</code>	query, location → list of POIs, geo, hours	Google Places
<code>search_flights</code>	origin, dest, dates, pax → flight options + price	Amadeus
<code>search_hotels</code>	location, dates, room needs → hotel + rate options	Booking.com
<code>search_activities</code>	location, category → tickets/tours	Klook
<code>check_visa</code>	nationality, destination → visa requirement, passport rule	Passport Index
<code>get_weather</code>	location, date range → forecast	OpenWeatherMap
<code>get_route</code>	origin, dest, mode → duration, distance	Mapbox

Tool-call rules: every tool call and its raw result is logged (`agent_logs`) for audit. Calls that are independent (e.g. flights + hotels + weather) run **in parallel**; calls that depend on a prior result (e.g. route depends on chosen hotel) run sequentially.

3.3. Skills (internal behaviors, not just APIs)

Beyond calling external tools, the agent has defined internal skills that govern *how* it behaves:

- **Research skill:** knows *what* to search, *how* to read raw API results, and *how* to rank and select the best options for this specific group — not just "call the API and paste the output" (see Section 3.4 for detail).
- **Clarification skill:** reads the group chat to extract what members have already stated (dates, budget, dietary needs, passport status, preferences); only asks a follow-up question when a specific piece of information is genuinely missing *and* required to proceed. Never runs a generic intake form; never asks for something already mentioned in the chat.
- **Conflict-mediation skill:** when preferences clash (beach vs. mountain) or dates don't overlap, probes individual members for underlying reasons and proposes a hybrid compromise instead of picking a side.
- **Local-repair skill:** when one leg is downvoted, patches only that leg in the existing itinerary object instead of regenerating the whole plan.
- **Validation skill:** before showing any tool result to users, checks it against basic sanity rules (price > 0, dates within trip window, place still open) and drops or re-queries invalid results.

3.4. Research Skill: How the Agent Finds Places, Hotels & Restaurants

Calling a tool is just one step. The Research skill is the *full sequence* the agent runs to turn raw API responses into a meaningful, group-tailored selection. It is not hardcoded per category — the agent applies the same reasoning pattern whether it is searching for a restaurant, a hotel, or a tourist attraction.

Step 1: Form a targeted query from group context

Before calling any tool, the agent reads the structured state and constructs a *specific* query, not a generic one.

- Bad (generic): `search_places("restaurant", "Da Nang")`
- Good (context-aware): `search_places("halal seafood restaurant large party Da Nang beach", location={lat, lng}, radius=2km, open_on="2025-04-12 19:00")`

The difference is that the good query encodes diet constraints, group size, location proximity to the chosen hotel, and the specific evening from the itinerary.

Step 2: Call the right tool(s), sometimes multiple

For some categories the agent needs more than one source to get a confident answer:

- **Hotels:** `search_hotels` (Booking.com) for price/availability + `search_places` (Google Places) for rating, photos and walking distance to attractions.
- **Restaurants:** `search_places` for rating/hours/cuisine + (optionally) a web search if the group wants a locally famous spot that APIs miss.
- **Attractions/activities:** `search_activities` (Klook) for ticketed venues + `search_places` for free or non-ticketed POIs.
- **Maps & routing:** `get_route` called once per consecutive leg pair to compute real transit time (walking, driving, taxi), not straight-line distance — because a hotel 1 km away but across a river can mean 20 minutes by road.

Step 3: Filter by hard constraints (automatic disqualifiers)

The agent drops any result that violates a hard constraint before scoring:

Constraint	Disqualifier
Budget	price per person > group budget cap
Diet	restaurant cannot accommodate a stated allergen or dietary rule
Capacity	venue does not seat the full party size
Timing	place is closed during the planned visit slot
Visa / legal	activity requires a permit the group cannot get in time

Step 4: Score and rank remaining candidates

After filtering, the agent scores each surviving candidate using a weighted formula derived from the group's stated priorities:

$$\text{score}(c) = w_1 \cdot \text{rating}(c) + w_2 \cdot \text{price_fit}(c) + w_3 \cdot \text{proximity}(c) + w_4 \cdot \text{preference_match}(c)$$

where weights w_1 – w_4 are inferred from what the group has said in the current chat session (e.g. a group that mentions “cheap” or “tight budget” gets high w_2 ; one that says “comfortable” or “don’t want to be tired” gets high w_1 and w_3). Weights default to equal if no signal is present. `preference_match` also checks the Feedback log: options similar to what the group already rejected this session are penalised.

Step 5: Select the top option and keep 2–3 alternatives

The highest-scoring candidate becomes the proposed leg. The next 2–3 become the instant-swap alternatives shown on the voting card, so when a member downvotes the main choice, the agent does *not* re-search from scratch — the alternatives are already scored and ready.

Step 6: Pin to the map

Each selected place is geocoded (lat/lng from the tool result or `search_places`) and added to the Mapbox layer so the group can visually verify the spatial arrangement of the itinerary — e.g. catching that the Day 2 restaurant and Day 2 attraction are on opposite sides of the city.

3.5. Destination Discovery: When the Group Does Not Know Where to Go

This is the harder case and the one most users actually face. The group says something like “*somewhere warm, 3 days, not too far from Hanoi, budget \$150/person, half the group wants to hike, the other half wants the beach*” — no specific destination yet.

The agent runs a two-stage process before any hotel or restaurant is searched:

Stage A: Generate candidate destinations (model reasoning, no API yet)

The agent reasons over the constraints purely from its own knowledge first, without calling any tool, because broad destination thinking is cheaper and faster as a language model inference step:

- Parse the soft signals: “warm” + “not far from Hanoi” + “hike + beach” → shortlist: Ha Long Bay, Ninh Binh + Cat Ba, Phong Nha, Da Nang.
- Apply hard filters immediately: trip duration 3 days → flight or drive $\leq 2\text{h}$ from Hanoi (otherwise transit eats the trip); budget \$150/person → drop international destinations.
- Conflict signal detected (hike vs. beach) → flag for Conflict-mediation skill, but also note that Cat Ba island (limestone karst + beach) could satisfy both — keep it as the synthesis candidate.

Output: a ranked shortlist of 3–5 destinations with a one-line rationale for each.

Stage B: Validate each candidate with real data

Now the agent calls tools — one parallel batch per candidate to check that the reasoning holds up against live data:

- `get_weather(destination, trip_dates)` — rule out candidates with bad weather on those dates (typhoon season, heavy rain).
- `search_flights(HAN, destination, dates, pax)` — check actual flight availability and price; if no flight fits the budget, drop the destination.
- `search_places(destination, "hike OR beach OR nature")` — confirm the activity types exist and are accessible (not seasonally closed).
- `check_visa(nationalities, destination)` — for international options.

Candidates that fail any check are dropped. Surviving candidates are re-scored and the top 2–3 are presented to the group as a **Destination Vote** before any detailed itinerary is planned. This avoids wasting research effort on a destination the group rejects.

Once a destination is chosen

The full Research Skill (Section 3.4) then runs on the confirmed destination: searching hotels, restaurants, and activities within it, forming context-aware queries, filtering and scoring — exactly as described above.

3.6. Place Selection Reasoning: Why the Agent Picks One Place Over Another

Scoring candidates (Section 3.4, Step 4) tells the agent *which individual place ranks highest*, but does not account for *how places interact with each other* in a full day. The agent runs an additional Place Selection Reasoning pass after individual scoring to make the final selection spatially and temporally coherent.

Cluster grouping

The agent groups all high-scoring candidates (hotels, restaurants, attractions) by geographic cluster before picking the combination to put in the itinerary. The goal is to minimise total daily travel rather than pick the highest-rated individual items regardless of location.

- Each candidate receives a cluster label based on its geo-coordinates (e.g. “Da Nang Beach North”, “Son Tra Peninsula”, “Da Nang City Centre”).
- The agent then selects a **cluster-coherent combination**: if the best-rated restaurant is 14 km from the best-rated morning attraction, but the second-rated restaurant is 400 m away, the agent picks the second restaurant — because the travel saved ($14\text{ km} \times \text{two trips} = 28\text{ km}$) outweighs the small rating difference.
- Hotels are selected first because they anchor the spatial layout of the day: a hotel in Cluster A makes all Cluster A places “close” and all Cluster B places “far”. The agent explicitly reasons about this anchor effect before committing to a hotel.

Half-day block planning

A day is split into natural blocks (Morning / Afternoon / Evening) rather than planning stop-by-stop from 8 am to 10 pm. The agent assigns a cluster to each block, so all stops within a block are geographically co-located.

Block	Cluster assigned	Agent reasoning
Morning	Son Tra Peninsula	outdoor activities best done early, before heat and crowds
Afternoon	Da Nang City Centre	museums/shopping open all day; also closest to hotel
Evening	Han River riverside	dinner + night market; walkable from City Centre

Switching clusters between blocks is acceptable; switching *within* a block triggers a warning and the agent tries to eliminate the cross-cluster hop before finalising.

Explicit anti-zigzag check

After assembling the full-day stop list, the agent computes the total travel distance and compares it against the convex hull of the same stops (the minimum possible travel if ordered optimally). If the ratio exceeds $1.6\times$, the day is flagged as inefficient and a nearest-neighbour reorder is applied on the flexible stops (fixed-time anchors are not moved).

3.7. Spatial Planning: Routing, Distance, Traffic & Time

After place selection, the agent runs a Spatial Planning pass to make the schedule physically feasible — checking that the group can actually get from A to B to C within the day’s time budget.

What the agent computes per day

For each day in the itinerary, the agent models the day as an ordered sequence of stops $S_1 \rightarrow S_2 \rightarrow \dots \rightarrow S_n$ and computes:

- Real road distance and travel time** via `get_route(origin, dest, mode, departure_time)`. The `departure_time` parameter is critical: the Mapbox Directions API returns *time-of-day-aware* travel time, so a 10-minute drive at 10 am becomes 35 minutes at 5 pm in a congested city. The agent always passes the actual planned departure time, not a generic estimate.
- Time budget per stop:** arrival time = departure from previous stop + transit time. The agent checks that `arrival_time + min_visit_duration ≤ closing_time` for every stop.
- Slack time:** the gap between end of one stop and start of next. If `slack < 15` minutes on any transition, the agent flags it as a tight connection and either compresses the visit at the earlier stop or proposes swapping the order.

Transport mode selection

The agent picks a transport mode based on distance and group size — not just the fastest option:

Distance	Default mode	Agent reasoning
< 1 km	Walking	Avoids traffic, free, no wait time.
1–5 km	Grab / taxi	Faster than walking, cheaper than private car for small group.
5–30 km	Private van (if ≥ 6 pax)	Taxi convoy for large groups is slower and costs more; a rented van keeps the group together.
> 30 km	Flight or intercity bus	Agent checks if a short domestic flight is cheaper than 5+ hours on a bus given the group’s time value.

The agent overrides this default if the group has mentioned a transport preference in the chat (e.g. “we want to rent motorbikes”) or if real pricing from `search_flights` makes a different mode clearly better.

Schedule re-ordering

If the initial stop order is spatially inefficient (e.g. the agent placed a lunch spot that requires backtracking 8 km), it applies a simple nearest-neighbour re-order on stops within the same half-day block, then re-checks all transit times. Fixed-time constraints (a museum that only has a 10 am slot, or a dinner reservation) are anchored and re-ordering only moves the flexible stops around them.

Traffic & congestion flags

For city legs (Hanoi, Ho Chi Minh City, Da Nang downtown), the agent knows that morning rush (7–9 am) and evening rush (5–7 pm) can double travel time. It:

- Avoids scheduling cross-city transitions during peak hours where possible.
- If unavoidable, pads the transit slot with a +50% time buffer and notes this explicitly on the itinerary card so the group is not surprised.
- Uses the `departure_time`-aware route call to get the actual congested estimate rather than guessing.

Meal timing and opening-hour awareness

Restaurants and attractions have hard open/close windows. The agent checks:

- Lunch slot is pinned to 11:30–13:30; dinner to 18:00–20:30. If the proposed lunch restaurant does not open until 11:00 and the previous attraction ends at 10:45, the agent inserts a 45-minute free buffer (e.g. walking the beach) rather than leaving a dead gap that confuses the group.
- For attractions with timed entry (e.g. Ba Na Hills cable car, museum slots), the agent anchors the entire day around that fixed time and plans all other stops backwards/forwards from it.
- If a must-see attraction closes at 17:00, the agent ensures the group arrives by 15:00 (not 16:45) to allow adequate visit time, and builds backwards from there.

Worked spatial example

Scenario: 6-person group, Da Nang, Day 2. Candidate stops after scoring: Marble Mountains (09:00 open), Bà Nà Hills (cable car 08:00–17:00, 35 km from city), My Khe Beach (free, 4 km from hotel), Han Market lunch (city centre, 11:00 open).

Agent spatial reasoning:

1. Bà Nà Hills is 35 km away and requires a full half-day minimum → anchor it to Morning block, depart hotel 07:30.
2. Marble Mountains (09:00 open, 10 km from city) conflicts with Bà Nà Hills in the Morning block → cannot do both. Agent places Marble Mountains in Afternoon on a different day.
3. Han Market for lunch: city centre, opens 11:00 → tight if returning from Bà Nà Hills by 13:00; agent pads transit to 12:30 and suggests lunch at a Bà Nà Hills on-site restaurant instead (saves 35 km return trip before lunch).
4. My Khe Beach: 4 km from hotel, no time constraint → Evening block, after group returns from Bà Nà Hills, sunset walk.

Result: Day 2 is Bà Nà Hills (all morning, lunch on-site) → return to city (Grab van, 40 min at 14:00, off-peak) → hotel rest → My Khe Beach sunset. Total travel: 90 min vs. the naïve order which would have been 180 min.

Output to the itinerary

Each leg in the final itinerary object carries: `{place_id, lat, lng, arrive_at, depart_at, transit_to_next: {mode, duration_min, distance_km, congestion_buffer_min, via}}`

This is what the Mapbox layer renders as the route line between stops, and what the PDF export uses to print “Grab van 40 min (peak: +20 min buffer) → Hotel” between timeline entries.

3.8. Agentic Reasoning Loop (not a fixed pipeline)

A common failure mode is describing an “agent” that is really just a hardcoded sequence of steps. TripMate AI’s agent does not execute Step 1 → 2 → 3 blindly; at every turn it runs a **Think** → **Act** → **Observe** loop over the current state:

1. **Think:** given the structured state (Section 3.5), decide the single best next action — call a tool, invoke a skill, ask the group a question, or produce the final proposal. This decision is made by the model itself (Pro for the decision, Flash for cheap sub-steps), not by an `if/else` script.
2. **Act:** execute that one action (e.g. call `search_hotels`, or run the Conflict-mediation skill).
3. **Observe:** read the result, update the structured state, and loop back to Think.
4. **Stop condition:** loop ends when the state is complete enough to propose a plan, or the group has reached consensus, or a retry/round cap (Section 3.7) is hit.

This means the order of actions is **decided at runtime, per group**, e.g.:

- If dates, budget and diet are already stated clearly in the first messages, the agent **skips** the Clarification skill entirely and goes straight to tool calls — it does not force a Q&A step just because the diagram has one.
- If `check_visa` unexpectedly returns “passport expires in 3 months”, the agent **inserts** a new clarifying question on the spot, even though the group already passed the “official” Q&A phase.
- Tool calls are chosen and ordered by **data dependency**, not by a fixed list: `get_route` only fires after a hotel is chosen; `search_flights`, `search_hotels` and `get_weather` fire together only when nothing later depends on their order.
- If preferences already agree (no conflict detected), the Conflict-mediation skill is never invoked.

The **5 UX phases** shown in Figure 1 (chat → Q&A → voting → lock → settle) are checkpoints exposed to the *frontend* so humans always know where they are in the process — they are not the agent’s internal execution order, which stays a dynamic loop underneath.

3.9. Context Management

Group chats can run to hundreds of messages, which will not fit (or is wasteful) in a single prompt every turn:

- The agent keeps a **running structured summary** (JSON: constraints and preferences stated in this session, decisions made so far) updated after each turn, instead of replaying full chat history. This summary is built fresh from the current trip’s chat — nothing is assumed from past sessions.
- Raw chat is stored in the database; only the structured summary + last *N* messages are sent to the model.
- The current itinerary is kept as a **stateful object** (list of legs with IDs) so votes and edits reference a leg ID, not free text — this is what makes local repair (Step above) possible.

3.10. Retry & Error Handling

- Each external tool call: up to **2 retries** with exponential backoff on timeout/5xx.
- If a provider is still failing, the agent falls back to **cached results** (Redis, TTL 6–24h) for that query and flags the data as “may be outdated” to the user.
- If a tool has no usable result at all, the agent tells the group plainly what it could not check, instead of guessing or hallucinating a price/availability.
- A Q&A loop is capped at **3 rounds**; if members don’t respond, the agent proceeds with reasonable defaults and marks assumptions in the proposal for the group to correct.

3.11. Learning Feedback Loop

Within a session, the agent adapts to the group’s revealed preferences rather than repeating the same mistakes:

- Every downvote is stored with its reason (price, location, style) in the session state.
- Before proposing an alternative, the agent checks this feedback log and excludes options matching a pattern the group already rejected (e.g. “too far from center”).

- Feedback is session-scoped: it accumulates within the current planning session and is discarded afterwards. Each new trip starts fresh — the agent does not assume a group’s preferences from a past trip carry over to this one.

3.12. Worked Example: the Loop in Action

Scenario A — group already knows the destination (Da Nang, 3 days, 6 pax):

- *Turn 1* — Think: “destination and dates stated in chat; budget not mentioned; dietary needs not mentioned but group has a halal member who stated it just now” → Act: Clarification skill detects budget is missing and needed; asks one question: “What is the rough per-person budget for this trip?” (diet already extracted from chat, no need to ask) → Observe: budget received, session state updated.
- *Turn 2* — Think: “state complete, run Research skill for hotels” → Act: form context-aware query (halal, 6 pax, beach-side, \$80/night cap), call `search_hotels` + `search_places` + `get_weather` in parallel; hotel API times out → retry, succeeds; filter by budget + capacity, score 8 candidates, pick top 1 + keep 2 alts → Observe: hotel and weather written to state.
- *Turn 3* — Think: “hotel anchored to Beach North cluster; now research and cluster-group all attractions/restaurants” → Act: Research skill fetches candidates; Place Selection Reasoning groups by cluster (Beach North, City Centre, Son Tra, Bà Nà Hills); detects Bà Nà Hills (35 km) conflicts with Marble Mountains (12 km from city) in same morning block → moves Marble Mountains to Day 3; assigns half-day blocks per cluster; anti-zigzag check passes → Observe: spatially coherent draft itinerary written to state.
- *Turn 4* — Think: “stops ordered, compute real transit times” → Act: call `get_route` with actual `departure_time` per leg (07:30 hotel → Bà Nà Hills: 40 min off-peak; 17:30 city return: 55 min peak + 50% buffer = 82 min flagged); pick Grab van for group of 6 (5–30 km rule) → Observe: transit times, modes and congestion buffers appended; map layer updated.
- *Turn 5* — Think: “all data present and valid” → Act: Validation skill, then stream Proposal Cards. Stop condition reached.
- *Turn 6 (later)* — Member downvotes Day 2 Lunch (too expensive). Think: “single leg rejected” → Act: Local-repair skill, check Feedback log (exclude “expensive” options), surface pre-scored alternatives → Observe: new option accepted. Loop ends for this leg.

Scenario B — group does not know where to go (“somewhere warm, near Hanoi, 3 days, hike + beach”):

- *Turn 1* — Think: “no destination, only soft signals” → Act: Destination Discovery Stage A — reason from model knowledge, produce shortlist: Ha Long Bay, Cat Ba, Ninh Binh + coast, Phong Nha. Flag hike/beach conflict, note Cat Ba as synthesis candidate → Observe: candidate list in state.
- *Turn 2* — Think: “must validate candidates with real data” → Act: Destination Discovery Stage B — call `get_weather` + `search_flights`(HAN, candidate, dates) in parallel for all 4 candidates; one candidate has typhoon risk → dropped; one flight is over budget → dropped; Cat Ba and Ninh Binh survive → Observe: 2 validated candidates.
- *Turn 3* — Think: “present Destination Vote to group” → Act: emit 2 Destination Cards with rationale → Observe: group votes Cat Ba.
- *Turn 4 onwards* — same as Scenario A (Research + Spatial Planning on Cat Ba).

No two groups take the same path through this loop — the sequences above are example runs, not scripts.

4. Backend Architecture

- **Runtime:** Python 3.11+, FastAPI (async, auto OpenAPI docs).
- **Database:** PostgreSQL + SQLAlchemy 2.0 (async) + Alembic migrations.
- **Cache/Broker:** Redis — caches tool results, rate-limits, WebSocket pub/sub.
- **Background jobs:** Celery / BackgroundTasks for PDF export (WeasyPrint) and email.

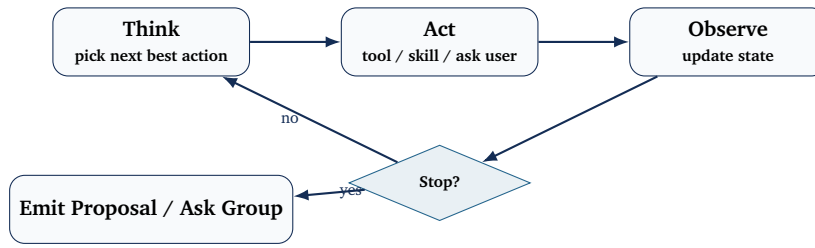


Figure 2: Internal Think-Act-Observe loop: the agent decides its own next action every turn, bounded only by the retry/round caps and stop condition (Section 3.7)

- **Settlement engine:** 3 tables (`expenses`, `expense_splits`, `settlements`); greedy matching on net balances $\text{Net}_u = \sum \text{Paid}_u - \sum \text{Owed}_u$ minimizes transfer count.

5. Frontend

React 18 + Vite + TypeScript, Tailwind CSS + shadcn/ui, Zustand for state, TanStack Query for caching/optimistic updates. Responsive across mobile, tablet, desktop.

6. Deployment & Security

- **Pipeline:** Docker Compose (local/staging), Vercel (frontend CDN), Render/VPS (backend), GitHub Actions (CI).
- **Security:** bcrypt password hashing, JWT access + Redis-rotated refresh tokens, RBAC, `bleach` sanitization, Pydantic v2 schema validation on all AI output.
- **Data privacy:** all trip-specific information (dietary needs, budget, passport details) lives only in the session chat and the agent's session state; it is not stored in permanent user profiles. Access is scoped by RBAC and not shared outside the room.
- **Rate limiting & audit:** Redis-backed limits on external and AI endpoints; all tool calls and token usage logged in `agent_logs`.

7. Expected Impact

- **Speed:** itinerary consensus in under 15 minutes instead of days of back-and-forth.
- **Fairness:** mathematically resolved schedule/preference conflicts, no single organizer carrying the load.
- **Reliability:** explicit retry, fallback and validation rules keep the agent from silently failing or hallucinating.

TripMate AI turns the chaos of group planning into a seamless, collaborative, and intelligent experience.